

ExamForge: Layered Style Architecture



Submitted by:

Mirza Zain Ali Baig 2025(S)-SE-20

Muhammad Sohaib Aslam 2025(S)-SE-13

Submitted to:

Sir Hassan

Department of Computer Science
University of Engineering and Technology Lahore, New
Campus

ExamForge: Layered Architecture

A test-paper generator built with Next.js, React, Supabase and Prisma/SQLite

Github Link: https://github.com/mirza-zain-ali-baig/University/tree/main/Semester4/SDA_LAB/Week03/examForge

ExamForge lets a teacher choose a class, subject, chapters and a blueprint (question types by difficulty), and then builds an exam paper from a question bank, from AI, or from both. The code is organised in four layers. Each layer talks only to the layer directly below it, so the interface, the rules and the storage can change independently.

Architecture Overview

1. Presentation Layer <i>User Interface</i>
2. Business Logic Layer <i>Rules and calculations</i>
3. Data Access Layer <i>Database communication</i>
4. Database Layer <i>Stores the actual data</i>

Requests flow down through the layers; results flow back up.

1. Presentation Layer (User Interface)

Everything the teacher sees and clicks. It never touches a database; it only calls the project's own /api endpoints through small fetch helpers.

- **App shell:** navigation between Dashboard, Generate Test, Question Bank, Analytics, History and My Tests.
- **Generator wizard:** collects class, subject, chapters, topics, blueprint, source (bank, AI or hybrid), variants and shuffle options.
- **Output views:** test preview and editing, print/export, and analytics and history charts.

Files: `src/app/page.tsx`, `src/components/test-gen/*`, `src/components/ui/*`

Technology: React 19, Next.js App Router, Tailwind CSS, shadcn/ui, Recharts

2. Business Logic Layer (Rules and Calculations)

The core of ExamForge: all exam-building rules live here. API routes validate input and coordinate the work; the library modules do the calculations.

- **Test generation:** `generateTest()` works through each blueprint cell (MCQ, Short, Long by Easy, Medium, Hard), picks questions at random and records any shortfall.
- **AI fill:** when the bank has too few questions, `generateAIQuestions()` asks the LLM for the rest.
- **Variants and shuffling:** `generateBatch()` creates up to 6 variants, reorders questions and MCQ options, and totals the marks.

- **Analysis and validation:** history, statistics, overlap and similarity checks; required fields and limits are validated.

Files: `src/lib/test-generator.ts`, `src/lib/ai-questions.ts`, `src/lib/types.ts`, `src/app/api/*`

3. Data Access Layer (Database Communication)

The only code that knows how to talk to storage. It turns function calls into queries and returns typed rows, hiding database details from the layers above.

- **supabase.ts:** creates the server-side Supabase client and defines the row types.
- **question-bank.ts:** reusable queries for classes, subjects, chapters, topics, questions and MCQ options, with paging past the 1,000-row limit.
- **db.ts:** a single Prisma client used to create, list, read and delete saved tests.

Files: `src/lib/supabase.ts`, `src/lib/question-bank.ts`, `src/lib/db.ts`

4. Database Layer (Stores the Actual Data)

ExamForge uses two data stores with different jobs.

- **Question bank (Supabase PostgreSQL, read-only):** classes, subjects, chapters, topics, questions, mcq_options, question_types and difficulty_levels.
- **Saved tests (SQLite via Prisma):** a GeneratedTest table holding the title, class, subject, marks, duration, source, timestamps and the full paper as JSON.

How the Layers Work Together

When a teacher generates a test, the wizard (Presentation) sends a request to `/api/generate-batch`. The route validates it and `generateBatch()` (Business Logic) selects questions, calling the AI if the bank falls short. The query functions (Data Access) fetch rows from Supabase (Database), and the finished paper returns to the preview screen. Saving the paper goes through Prisma into SQLite.

Note: a few API routes (stats, curriculum, questions, history, tests) query Supabase or Prisma directly instead of using a function in `src/lib`. Moving them there would make the layering fully strict.

Example: Generating a Test, Layer by Layer

1. Presentation	Teacher completes the wizard; the browser sends POST <code>/api/generate-batch</code> with the blueprint.
2. Business Logic	The route validates the request, then <code>generateBatch()</code> selects questions, calls the AI if the bank falls short, and shuffles and totals marks.
3. Data Access	<code>question-bank.ts</code> and <code>supabase.ts</code> query questions, options and lookups; later POST <code>/api/tests</code> saves the paper through Prisma.
4. Database	Supabase returns the matching rows; SQLite stores the saved paper. The result travels back up to the preview screen.